OGC® DOCUMENT: 26-007

External identifier of this OGC® document: http://www.opengis.net/doc/IS/indoorjson/1.0

Open Geospatial Consortium

OGC INDOORGML 2.0 PART 2B – JSON ENCODING

STANDARD
Implementation

CANDIDATE SWG DRAFT

Version: 1.0.0
Submission Date: 2026-02-28
Approval Date: 2099-01-01
Publication Date: 2099-01-01
Editor: Taehoon Kim, Abdoulaye Diakite, Ki-Joune Li, Sisi Zlatanova

License Agreement

Use of this document is subject to the license agreement at https://www.ogc.org/license

Suggested additions, changes and comments on this document are welcome and encouraged. Such suggestions may be submitted using the online change request form on OGC web site: http://ogc.standardstracker.org/

Copyright notice

Note

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. The Open Geospatial Consortium shall not be held responsible for identifying any or all such patent rights.

Recipients of this document are requested to submit, with their comments, notification of any relevant patent claims or other intellectual property rights of which they may be aware that might be infringed by any implementation of the standard set forth in this document, and to provide supporting documentation.

CONTENTS

LIST OF TABLES

LIST OF NORMATIVE STATEMENTS

I ABSTRACT

The IndoorGML 2.0 Part 2b – JSON Encoding Standard presents the implementation-dependent JavaScript Object Notation (JSON) encoding of the concepts defined by the IndoorGML 2.0 Part 1 – Conceptual Model Standard (OGC 22-045r5,OGC 22-045r5). The JSON Encoding is compliant to JSON schema specified by RFC8259. The IndoorGML JSON (IndoorJSON) encoding defines a concrete implementation schema for representing indoor spatial information, including primal space, dual space, thematic layers, interlayer connections, and navigation-related indoor features. This encoding can be used to store and exchange 2D and/or 3D indoor space and indoor navigation network models in the JSON format as data sets or via web services.

This Part 2b of the Standard defines how the concepts developed in Part 1 are realized using JSON schemas. The IndoorJSON Encoding represents a full mapping of the Conceptual Model and is derived fully automatically from the UML model following the UML-to-JSON encoding rules as defined by OGC 24-017r1. The IndoorJSON adopts JSON Schema as its primary schema language and uses geometry objects compatible with OGC Features and Geometries JSON (JSON-FG, OGC 21-045r1), where appropriate. This design choice addresses the limitation that conventional GeoJSON does not provide sufficient support for indoor 3D geometry, especially solid and volumetric geometry. Table 1 maps requirements classes from the IndoorGML 2.0 Conceptual Model into the implementation details documented in this standard.

Table 1

CONCEPTUAL MODEL	SECTION	JSON SCHEMA
IndoorGML Core	[section_6.1]	[core.json]
Navigation Extension	[section_6.2]	[navi.json]

II KEYWORDS

The following are keywords to be used by search engines and document catalogues.

OGCDoc, OGC documents, indoor, navigation, IndoorGML, JSON, GeoJSON, JSON-FG

PREFACE

In order to achieve consensus on the modeling indoor spaces and their neighborhood relationships to support indoor location-based services, a UML Conceptual Model, IndoorGML 2.0 Part 1, was approved as an OGC Standard (OGC 22-045r5,OGC 22-045r5) in June, 2025. This model covers geometric and semantic properties of indoor spaces relevant for indoor navigation. This IndoorGML 2.0 Part 2: JSON Encoding (IndoorJSON) Standard defines how those concepts should be realized using JSON format. The IndoorJSON encoding is intended for software developers, data providers, standards implementers, and service providers who need a lightweight, web-friendly, and machine-readable representation of IndoorGML datasets.

IV SECURITY CONSIDERATIONS

No security considerations have been made for this Standard.

V SUBMITTING ORGANIZATIONS

The following organizations submitted this Document to the Open Geospatial Consortium (OGC):

- The University of New South Wales
- Pusan National University
- CityGeometrix
- National Institute of Advanced Industrial Science and Technology

VI SUBMITTERS

All questions regarding this submission should be directed to the editor or the submitters:

NAME	AFFILIATION	OGC MEMBER
Taehoon KIM	National Institute of Advanced Industrial Science and Technology	Yes
Abdoulaye Diakite	CityGeometrix	Yes
Ki-Joune Li	Pusan National University	Yes
Sisi Zlatanova	The University of New South Wales	Yes

1 SCOPE

1 SCOPE

The OGC IndoorGML 2.0 Part 2b – JSON Encoding Standard defines a JSON encoding for IndoorGML 2.0 Part 1 — Conceptual Model. The encoding is referred to as IndoorJSON.

The IndoorJSON Standard specifies:

- JSON schema for IndooGML 2.0 core module;
- JSON schema for IndoorGML 2.0 navigation module;
- the geometry encoding rules based on JSON-FG geometry objects; and
- a statement to which IndoorJSON conformance classes a IndoorGML 2.0 conforms to.

IndoorJSON is an implementation encoding of IndoorGML 2.0 Part 1. Therefore, IndoorJSON instances shall be interpreted according to the semantics defined in the IndoorGML 2.0 conceptual model.

2 CONFORMANCE

2 CONFORMANCE

This standard defines implementation specification which specifies how the IndoorGML 2.0 Conceptual Model should be implemented using JavaScript Object Notation (JSON). The Standardization Target for this standard is:

- Implementations of the IndoorGML 2.0 Conceptual Model using JSON encodings.

Conformance with this standard shall be checked using all the relevant tests specified in Annex A (normative) of this document. The framework, concepts, and methodology for testing, and the criteria to be achieved to claim conformance are specified in the OGC Compliance Testing Policies and Procedures and the OGC Compliance Testing web site.

In order to conform to this OGC® interface standard, a software implementation shall choose to implement:

- Any one of the conformance levels specified in Annex A (normative).

All requirements-classes and conformance-classes described in this document are owned by the standard(s) identified.

Table 2

Conformance class	URI
IndoorGML Core	
Navigation Extension	

3 NORMATIVE REFERENCES

3 NORMATIVE REFERENCES

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

Johannes Echterhoff: OGC 24-017r1, *Best Practice for OGC — UML to JSON Encoding Rules*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/BP/uml-to-json-encoding/1.0.

Sisi Zlatanova, Abdoulaye Diakite, Taehoon Kim, Ki-Joune Li: OGC 22-045r5, *OGC IndoorGML 2.0 Part 1 – Conceptual Model*. Open Geospatial Consortium (2025). http://www.opengis.net/doc/IS/indoorgml/2.0.

Clemens Portele, Panagiotis (Peter) A. Vretanos: OGC 21-045r1, *OGC Features and Geometries JSON — Part 1: Core*. Open Geospatial Consortium (2026). http://www.opengis.net/doc/IS/json-fg-1/1.0.

ISO: ISO 19101-1, *Geographic information — Reference model — Part 1: Fundamentals*. International Organization for Standardization, Geneva https://www.iso.org/standard/59164.html.

ISO: ISO 19107, *Geographic information — Spatial schema*. International Organization for Standardization, Geneva https://www.iso.org/standard/66175.html.

T. Bray (ed.): IETF RFC 8259, *The JavaScript Object Notation (JSON) Data Interchange Format*. RFC Publisher (2017). https://www.rfc-editor.org/info/rfc8259.

4 TERMS AND DEFINITIONS

4 TERMS AND DEFINITIONS

This document uses the terms defined in OGC Policy Directive 49, which is based on the ISO/IEC Directives, Part 2, Rules for the structure and drafting of International Standards. In particular, the word “shall” (not “must”) is the verb form used to indicate a requirement to be strictly followed to conform to this document and OGC documents do not use the equivalent phrases in the ISO/IEC Directives, Part 2.

This document also uses terms defined in the OGC Standard for Modular specifications (OGC 08-131r3), also known as the ‘ModSpec’. The definitions of terms such as standard, specification, requirement, and conformance test are provided in the ModSpec.

For the purposes of this document, the following additional terms and definitions apply.

4.1. conceptual model

model that defines concepts of a universe of discourse

[**SOURCE:** ISO 19101-1]

4.2. conceptual schema

formal description of a [conceptual_model]

[**SOURCE:** ISO 19101-1]

4.3. feature

abstraction of real world phenomena

Note 1 to entry: Features are objects that have an identity that allows for distinguishing features from each other. Features can have spatial and non-spatial properties that describe the features in more detail. Spatial properties are properties that have a geometry from ISO 19107 as data type.

Note 2 to entry: Features are denoted by the stereotype «FeatureType» in the IndoorGML 2.0 Conceptual Model.

[**SOURCE:** ISO 19101-1]

5 CONVENTIONS

5 CONVENTIONS

5.1. Identifiers

The normative provisions in this standard are denoted by the URI

`http://www.opengis.net/spec/indoorgml-2/2.0/json`

All requirements and conformance tests that appear in this document are denoted by partial URIs which are relative to this base.

5.2. Use of JSON terminology

The following terms are used as specified in JSON. In particular:

- “member” refers to a name/value pair of an object;
- “key” refers to the name of a member; and
- “value” refers to the value of a member, which may be an object, array, string, number, boolean, or `null`.

For example, the GeoJSON “geometry” member has the key “geometry” and one of the GeoJSON geometries (or `null`) as its value.

6

INDOORJSON OBJECT

6 INDOORJSON OBJECT

REQUIREMENTS CLASS 1	
IDENTIFIER	`http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
TARGET TYPE	Implementation Specification
NORMATIVE STATEMENTS	Requirement 1: `/req/indoorjson` Requirement 1-7: `/req/dualspace` Requirement 2: `/req/indoorfeature` Requirement 3: `/req/thematiclayer` Requirement 4: `/req/primalspacelayer` Requirement 5: `/req/cellspace` Requirement 6: `/req/cellboundary` Requirement 8: `/req/node` Requirement 9: `/req/edge` Requirement 10: `/req/interlayerconnection`

IndoorJSON object represents one of feature class of IndoorGML 2.0 conceptual model.

The IndoorJSON Object:

- **must** have one member with the name `id`. The value must be a string that serves as a unique identifier for the IndoorJSON object and can be used to represent cross-references between IndoorJSON objects.
- **must** have one member with the name `featuretype`. The value is one of the possibilities in the figure [fig-core-indoorgml].

REQUIREMENT 1: REQUIREMENT FOR INDOORJSON OBJECT	
IDENTIFIER	`/req/indoorjson`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	An IndoorJSON object SHALL contain `id` and `featuretype` members.

6.1. IndoorFeatures

`IndoorFeature` object is an instance of IndoorFeature, the container for the indoor dataset. It aggregates one or more thematic layers and may include inter-layer connections.

The **`IndoorFeature`** object shall contain:

- `id`
- `featureType`
- `layers` for representing **`ThematicLayer`** object as item
- optionally, `layerConnections` for **`InterLayerConnection`** object as item

```
{
  "IndoorFeatures": {
    "$anchor": "IndoorFeatures",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["IndoorFeatures"] },
      "layers": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/ThematicLayer" },
        "uniqueItems": true
      },
      "layerConnections": {
        "type": "array",
        "items": { "$ref": "#/$defs/InterLayerConnection" },
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","layers"]
  }
}
```

Listing 1 — A JSON schema of the IndoorFeature:

```
{
  "featureType": "IndoorFeatures",
  "layers": [
    {
      "id": "TL-1",
      "featureType": "ThematicLayer",
      "semanticExtension": false,
      "theme": "Physical",
      "primalSpace": {...},
      "dualSpace": {...}
    },
    ...
  ],
  "layerConnections": [
    {
      "id": "ILC-1",
      "featureType": "InterLayerConnection",
```

```
      "connectedLayers": ["TL-1", "TL-2"],
      "connectedNodes": ["N1", "N101"]
    }
  ]
}
```

Listing 2 — An example of the IndoorFeature object:

REQUIREMENT 2: INDOORFEATURES SCHEMA	
IDENTIFIER	`/req/indoorfeature`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	An **`IndoorFeatures`** object SHALL contain at least one **`ThematicLayer`** object as item of `layers` value.

6.2. ThematicLayer

`ThematicLayer` object is an instance of ThematicLayer, which represents one contextual decomposition of indoor space. For example, physical layer, navigation layer, sensor layer, etc.

The **`ThematicLayer`** object shall contain:

- `id`
- `featureType`
- `semanticExtention` indicates whether there is extension module with additional semantic information associated to the **`PrimalSpaceLayer`**
- `theme` determines what type of model representation can be expected in the **`PrimalSpaceLayer`**
- optionally, `primalSpace` for **`PrimalSpaceLayer`** object
- optionally, `dualSpace` for **`DualSpaceLayer`** object

The core schema defines `semanticExtension` and `theme` members, with `theme` values including “Virtual”, “Physical”, “Tags”, and “Unknown”, according to the conceptual model.

```
{
  "ThematicLayer": {
    "$anchor": "ThematicLayer",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["ThematicLayer"] },
      "semanticExtension": { "type": "boolean" },
```

```
      "theme": { "enum": ["Virtual", "Physical", "Tags", "Unknown"] },
      "primalSpace": { "$ref": "#/$defs/PrimalSpaceLayer" },
      "dualSpace": { "$ref": "#/$defs/DualSpaceLayer" }
    },
    "required": ["id","featureType","semanticExtension","theme"]
  }
}
```

Listing 3 — A JSON schema of the ThematicLayer:

```
{
  "id": "TL-1",
  "featureType": "ThematicLayer",
  "semanticExtension": false,
  "theme": "Physical",
  "primalSpace": {...},
  "dualSpace": {...}
}
```

Listing 4 — An example of the ThematicLayer object:

REQUIREMENT 3: THEMATICLAYER SCHEMA	
IDENTIFIER	`/req/thematiclayer`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	A **`ThematicLayer`** object SHALL declare whether there is an extension module with additional semantic information using the `semanticExtension` member.
B	A **`ThematicLayer`** object SHALL determine what type of model representation can be expected using the `theme` member.
C	The `Theme` value SHALL be set to “Virtual”, “Physical”, “Tags”, and “Unknown”.

6.3. PrimalSpaceLayer

`PrimalSpaceLayer` object is an instance of PrimalSpaceLayer, which contains spatial objects representing indoor subdivisions.

The **`PrimalSpaceLayer`** object shall contain:

- `id`
- `featureType`
- `cellSpaceMember` for representing **`CellSpace`** object as item
- optionally, `cellBoundaryMember` for representing **`CellSBoundary`** object as item

- optionally, `creationDatetime`
- optionally, `terminationDatetime`

```
{
  "PrimalSpaceLayer": {
    "$anchor": "PrimalSpaceLayer",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["PrimalSpaceLayer"] },
      "creationDatetime": { "type": "string", "format": "date-time" },
      "terminationDatetime": { "type": "string", "format": "date-time" },
      "cellSpaceMember": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/CellSpace" },
        "uniqueItems": true
      },
      "cellBoundaryMember": {
        "type": "array",
        "items": { "$ref": "#/$defs/CellBoundary" },
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","cellSpaceMember"]
  }
}
```

Listing 5 — A JSON schema of the PrimalSpaceLayer:

```
{
  "primalSpace": {
    "id": "PS-1",
    "featureType": "PrimalSpaceLayer",
    "creationDatetime": "2025-10-30T13:00:00",
    "cellSpaceMember": [...],
    "cellBoundaryMember": [...]
  }
}
```

Listing 6 — An example of the PrimalSpaceLayer object:

REQUIREMENT 4: PRIMALSPACELAYER SCHEMA	
IDENTIFIER	`/req/primalspacelayer`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	A `PrimalSpaceLayer` SHALL contain one or more CellSpace objects.

6.4. CellSpace

CellSpace object is an instance of CellSpace, which represents a unit of indoor space. The **CellSpace** may correspond to a room, corridor, stairwell, elevator cabin, platform, zone, virtual partition, or other spatial subdivision.

An **CellSpace** object shall contain:

- `id`
- `featureType`
- `poi` is a flag that specifies whether this **CellSpace** is a point of interset
- optionally, `cellSpaceName` is the name assigned to the space according to internal convention
- optionally, `level`
- optionally, `duality` can be mapped to the `id` value of the **Node** object
- optionally, `cellSpaceGeom`
- optionally, `boundedBy` can have the `id` value of the linked **CellBoundary** objects
- optionally, `externalReference` for representing **ExternalReference** object

A **CellSpace** can be linked to a **Node** through the `duality` member. A **CellSpace** can be linked to a set of **CellBoundaries** through the `boundedBy` member. For `cellsSpaceGeom` member, it defines `cellSpaceGeom.geometry2D` using a `JSON-FG Polygon` geometry and `cellSpaceGeom.geometry3D` using a `JSON-FG Polyhedron` geometry. The `externalReference` member is used for the reference of an object to its corresponding object in an external data set.

```
{
  "CellSpace": {
    "$anchor": "CellSpace",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["CellSpace"] },
      "cellSpaceName:": { "type": "string" },
      "level": { "type": "string" },
      "poi": { "type":  "boolean" },
      "duality" : { "type": "string", "format": "uri-reference" },
      "cellSpaceGeom" : {
        "type": "object",
        "properties": {
          "geometry2D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polygon" },
          "geometry3D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polyhedron" }
        }
      },
      "boundedBy": {
```

```
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "uniqueItems": true
      },
      "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
    },
    "required": ["id","featureType","poi"]
  }
}
```

Listing 7 — A JSON schema of the CellSpace:

```
{
  "id": "C1",
  "featureType": "CellSpace",
  "duality": "TL-1:DS-1:N1",
  "poi": false,
  "level": "1",
  "cellSpaceGeom": {
    "geometry2D": {
      "type": "Polygon",
      "coordinates": [[[0,0], [2,0], [2,2], [0,2], [0,0]]]
    }
  },
  "boundedBy": ["TL-1:PS-1:B1"]
}
```

Listing 8 — An example of the CellSpace object:

REQUIREMENT 5: CELLSPACE SCHEMA	
IDENTIFIER	`/req/cellspace`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	A **`CellSpace`** object SHALL declare whether it is a point of interest space using the `poi` member.

6.5. CellBoundary

`CellBoundary` object is an instance of CellBoundary, a boundary of a CellSpace. A boundary may be physical or virtual.

The **`CellBoundary`** object shall contain:

- `id`
- `featureType`
- `isVirtual` indicate whether a **`CellBoundary`** corresponds to a virtual surface (true) or a physical one (false)

- optionally, `duality` can be mapped to the `id` value of the **`Edge`** object
- optionally, `cellBoundaryGeom`
- optionally, `externalReference` for representing **`ExternalReference`** object

A **`CellBoundary`** can be linked to a **`Edge`** through the `duality` member. For `cellBoundaryGeom` member, it defines `cellBoundaryGeom.geometry2D` using a `JSON-FG LineString` geometry and `cellBoundaryGeom.geometry3D` using a `JSON-FG Polygon` geometry. The `externalReference` member is used for the reference of an object to its corresponding object in an external data set

```
{
  "CellBoundary": {
    "$anchor": "CellBoundary",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["CellBoundary"] },
      "isVirtual": { "type":  "boolean" },
      "duality" : { "type": "string", "format": "uri-reference" },
      "cellBoundaryGeom" : {
        "type": "object",
        "properties": {
          "geometry2D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/LineString" },
          "geometry3D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polygon" }
        }
      },
      "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
    },
    "required": ["id","featureType","isVirtual"]
  }
}
```

Listing 9 — A JSON schema of the CellBoundary:

```
{
  "id": "B1",
  "featureType": "CellBoundary",
  "isVirtual": false,
  "duality": "TL-1:DS-1:E1",
  "cellBoundaryGeom": {
    "geometry2D": {
      "type": "LineString",
      "coordinates": [[2,0],[2,2]]
    }
  }
}
```

Listing 10 — An example of the CellBoundary object:

REQUIREMENT 6: CELLBOUNDARY SCHEMA	
IDENTIFIER	`/req/cellboundary`

REQUIREMENT 6: CELLBOUNDARY SCHEMA	
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	A **`CellBoundary`** object SHALL specify whether it is virtual using the `isVirtual` member.

6.6. DualSpaceLayer

`DualSpaceLayer` object is an instance of DualSpaceLayer, represents the graph structure corresponding to the primal space layer. It contains nodes and edges.

The **`DualSpaceLayer`** object shall contain:

- `id`
- `featureType`
- `isLogical` indicates whether the provided graph is a geometric or a logical
- `isDirected` specifies whether the provided graph is directed
- `nodeMember` for representing **`Node`** object as item
- optionally, `edgeMember` for representing **`Edge`** object as item
- optionally, `creationDatetime`
- optionally, `terminationDatetime`

```
{
  "DualSpaceLayer": {
    "$anchor": "DualSpaceLayer",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["DualSpaceLayer"] },
      "creationDatetime": { "type": "string", "format": "date-time" },
      "terminationDatetime": { "type": "string", "format": "date-time" },
      "isLogical": { "type": "boolean" },
      "isDirected": { "type": "boolean" },
      "nodeMember": {
        "type": "array",
        "minItems": 1,
        "items": { "$ref": "#/$defs/Node" },
        "uniqueItems": true
      },
      "edgeMember": {
        "type": "array",
        "items": { "$ref": "#/$defs/Edge" },
        "uniqueItems": true
      }
```

```
    },
    "required": ["id","featureType","isLogical","isDirected","nodeMember"]
  }
}
```

Listing 11 — A JSON schema of the DualSpaceLayer:

```
{
  "dualSpace": {
    "id": "DS-1",
    "featureType": "DualSpaceLayer",
    "creationDatetime": "2025-10-30T13:00:00",
    "isLogical": false,
    "isDirected": false,
    "nodeMember": [...],
    "edgeMember": [...]
  }
}
```

Listing 12 — An example of the DualSpaceLayer object:

REQUIREMENT 7: DUALSPACELAYER SCHEMA	
IDENTIFIER	`/req/dualspacelayer`
A	A **`DualSpaceLayer`** object SHALL specify whether the provided graph is a geometric or a logical using the `isLogical` member.
B	A **`DualSpaceLayer`** object SHALL specify whether the provided graph is directed using the `isLogical` member.
C	A **`DualSpaceLayer`** object SHALL contain one or more **`Node`** objects.

6.7. Node

`Node` object is an instance of Node, a state or a graph vertex in the dual space layer.

The **`Node`** object shall contain:

- `id`
- `featureType`
- `duality` must be mapped to the `id` member of the **`CellSpace`** object
- optionally, `geometry`
- optionally, `connects` can have the `id` member of the connected **`Edge`** object

A **Node** may be linked to a **CellSpace** through the `duality` member. A **Node** may be connects to a set of **Edges** through the `connects` member. For `geometry` member referred `JSON-FG Point` geometry.

```
{
  "Node": {
    "$anchor": "Node",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["Node"] },
      "duality" : { "type": "string", "format": "uri-reference" },
      "geometry" : { "$ref": "https://beta.schemas.opengis.net/json-fg/geometry-
object.json#/$defs/Point" },
      "connects" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","duality"]
  }
}
```

Listing 13 — A JSON schema of the Node:

```
{
  "id": "N1",
  "featureType": "Node",
  "duality": "TL-1:PS-1:C1",
  "geometry": {
    "type": "Point",
    "coordinates": [1,1]
  },
  "connects": ["TL-1:DS-1:E1"]
}
```

Listing 14 — An example of the Node object:

REQUIREMENT 8: NODE SCHEMA	
IDENTIFIER	`/req/node`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	A **Node** object SHALL contain a `duality` value by referencing the `id` member of to correspond **CellSpace**.

6.8. Edge

`Edge` object is an instance of Edge, which represents a relationship between two nodes. It may represent adjacency, connectivity, accessibility, or navigability.

The `Edge` object shall contain:

- `id`
- `featureType`
- `weight` can be used in graph-based applications
- `connects` must have the `id` value of the connected **`Nodes`** object
- optionally, `duality` can be mapped to the `id` value of the **`CellBoundary`** object
- optionally, `geometry`

An `Edge` must reference the connected **`Nodes`** using the `connects` member. An `Edge` can be linked to a **`CellBoundary`** through the `duality` member. For `geometry` member referred `JSON-FG LineString` geometry.

```
{
  "Edge": {
    "$anchor": "Edge",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["Edge"] },
      "duality" : { "type": "string", "format": "uri-reference" },
      "weight" : { "type": "number" },
      "geometry" : { "$ref": "https://beta.schemas.opengis.net/json-fg/geometry-
object.json#/$defs/LineString" },
      "connects" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      }
    },
    "required": ["id","featureType","weight","connects"]
  }
}
```

Listing 15 — A JSON schema of an Edge:

```
{
  "id": "E1",
  "featureType": "Edge",
  "duality": "B1",
  "weight": 1.0,
  "geometry": {
    "type": "LineString",
    "coordinates": [[1,1],[3,1]]
```

```
  },
  "connects": [
    "TL-1:DS-1:N1",
    "TL-1:DS-1:N2"
  ]
}
```

Listing 16 — An example of the Edge object:

REQUIREMENT 9: EDGE SCHEMA	
IDENTIFIER	`/req/edge`
INCLUDED IN	Requirements class 1: `http://www.opengis.net/spec/indoorgml-2/2.0/json/req`
A	An **`Edge`** object SHALL reference the connected Nodes using the `connects` member.
B	An **`Edge`** object SHALL contain a `weight` value.

6.9. InterLayerConnection

`InterLayerConnection` object is an instance of InterLayerConnection, represents a relationship between objects in different thematic layers.

The **`InterLayerConnection`** object shall contain:

- `id`
- `featureType`
- `connectedLayers` must be mapped to the `id` value of the two **`ThematicLayer`** objects
- `typeOfTopoExpression`
- optionally, `connectedNodes` can be mapped to the `id` value of the two **`Node`** objects
- optionally, `connectedCells` can be mapped to the `id` value of the two **`CellSpace`** objects
- optionally, `comment`

The `typeOfTopoExpression` member represents the topological relationship between two layers. The `typeOfTopoExpression` values including “CONTAINS”, “OVERLAPS”, “EQUALS”, “WITHIN”, “CROSSES”, and “OTHERS”, according to the conceptual model. Those topological values are in the form of verbs for which the subject is the first instance of the `connectedLayers` member. The values of the `connectedLayers`, `connectedNodes`, and `connectedCells` members are ordered and can be mapped to the same index.

```
{
  "InterLayerConnection": {
    "$anchor": "InterLayerConnection",
    "type": "object",
    "properties": {
      "id": { "type": "string" },
      "featureType": { "enum": ["InterLayerConnection"] },
      "connectedLayers": {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      },
      "connectedNodes" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      },
      "connectedCells" : {
        "type": "array",
        "items": { "type": "string", "format": "uri-reference" },
        "minItems": 2,
        "maximum": 2,
        "uniqueItems": true
      },
      "typeOfTopoExpression" : {
        "enum": ["CONTAINS", "OVERLAPS", "EQUALS", "WITHIN", "CROSSES", "OTHERS"]
      },
      "comment" : { "type": "string" }
    },
    "required": ["id","featureType","connectedLayers","typeOfTopoExpression"]
  }
}
```

Listing 17 — A JSON schema of the InterLayerConnection:

```
{
  "id": "ILC-1",
  "featureType": "InterLayerConnection",
  "connectedLayers": ["TL-1", "TL-2"],
  "connectedNodes": ["N1", "N101"]
}
```

Listing 18 — An example of the InterLayerConnection object:

REQUIREMENT 10: INTERLAYERCONNECTION SCHEMA	
IDENTIFIER	/req/interlayerconnection
INCLUDED IN	Requirements class 1: http://www.opengis.net/spec/indoorgml-2/2.0/json/req

6.10. ExternalReferenceType

The `externalReferenceType` object is used for the reference of an object to its corresponding object in an external data set. This is not a feature class of IndoorGML 2.0.

```
{
  "ExternalReferenceType": {
    "$anchor": "ExternalReferenceType",
    "type": "object",
    "properties": {
      "externalObject": {
        "type": "object",
        "properties": {
          "name": { "type": "string" },
          "uri": { "type": "string", "format": "uri" }
        },
        "required": ["name","uri"]
      },
      "informationSystem": { "type": "string", "format": "uri" }
    },
    "required": ["externalObject", "informationSystem"]
  }
}
```

Listing 19 — A JSON schema of the ExternalReferenceType:

A

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

A

ANNEX A (NORMATIVE) CONFORMANCE CLASS ABSTRACT TEST SUITE

NOTE: Ensure that there is a conformance class for each requirements class and a test for each requirement (identified by requirement name and number)

A.1. Conformance Class A

Example

REQUIREMENT A.1	
STATEMENT	conf/service/profile/basic-wps

RECOMMENDATION A.1	
STATEMENT	Verify that the server implements the Basic WPS conformance class.

REQUIREMENT A.2	
STATEMENT	Verify that the server implements the Synchronous WPS and/or the Asynchronous WPS conformance class. Verify that the requests and responses to a supported operation are syntactically correct. Verify that the service supports the Synchronous WPS Conformance class, the Asynchronous WPS Conformance class or both. Verify that all process offerings implement the native process model. Verify the following list of conformance tests: • Conformance test 1 • Conformance test 2

B

ANNEX B (NORMATIVE) SCHEMAS

B ANNEX B (NORMATIVE) SCHEMAS

B.1. JSON Schema of a IndoorJSON Core

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://www.opengis.net/spec/indoorgml/2.0/json/indoorjson.core.schema.
json",
  "title": "IndoorJSON for IndoorGML 2.0 core part",
  "description": "IndoorJSON Core",
  "oneOf": [
    {"$ref": "#/$defs/IndoorFeatures"       },
    {"$ref": "#/$defs/ThematicLayer"        },
    {"$ref": "#/$defs/InterLayerConnection" },
    {"$ref": "#/$defs/PrimalSpaceLayer"     },
    {"$ref": "#/$defs/CellSpace"            },
    {"$ref": "#/$defs/CellBoundary"         },
    {"$ref": "#/$defs/DualSpaceLayer"       },
    {"$ref": "#/$defs/Node"                 },
    {"$ref": "#/$defs/Edge"                 }
  ],
  "$defs": {

    "IndoorFeatures": {
      "$anchor": "IndoorFeatures",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["IndoorFeatures"] },
        "layers": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/ThematicLayer" },
          "uniqueItems": true
        },
        "layerConnections": {
          "type": "array",
          "items": { "$ref": "#/$defs/InterLayerConnection" },
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","layers"]
    },

    "ThematicLayer": {
      "$anchor": "ThematicLayer",
```

```
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["ThematicLayer"] },
        "semanticExtension": { "type": "boolean" },
        "theme": { "enum": ["Virtual", "Physical", "Tags", "Unknown"] },
        "primalSpace": { "$ref": "#/$defs/PrimalSpaceLayer" },
        "dualSpace": { "$ref": "#/$defs/DualSpaceLayer" }
      },
      "required": ["id","featureType","semanticExtension","theme"]
    },

    "PrimalSpaceLayer": {
      "$anchor": "PrimalSpaceLayer",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["PrimalSpaceLayer"] },
        "creationDatetime": { "type": "string", "format": "date-time" },
        "terminationDatetime": { "type": "string", "format": "date-time" },
        "cellSpaceMember": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/CellSpace" },
          "uniqueItems": true
        },
        "cellBoundaryMember": {
          "type": "array",
          "items": { "$ref": "#/$defs/CellBoundary" },
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","cellSpaceMember"]
    },

    "CellSpace": {
      "$anchor": "CellSpace",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["CellSpace"] },
        "cellSpaceName:": { "type": "string" },
        "level": { "type": "string" },
        "poi": { "type":  "boolean" },
        "duality" : { "type": "string", "format": "uri-reference" },
        "cellSpaceGeom" : {
          "type": "object",
          "properties": {
            "geometry2D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polygon" },
            "geometry3D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polyhedron" }
          }
        },
        "boundedBy": {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "uniqueItems": true
        },
        "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
      },
      "required": ["id","featureType","poi"]
    },
```

```
    "CellBoundary": {
      "$anchor": "CellBoundary",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["CellBoundary"] },
        "isVirtual": { "type":  "boolean" },
        "duality" : { "type": "string", "format": "uri-reference" },
        "cellBoundaryGeom" : {
          "type": "object",
          "properties": {
            "geometry2D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/LineString" },
            "geometry3D": { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Polygon" }
          }
        },
        "externalReference": { "$ref": "#/$defs/ExternalReferenceType" }
      },
      "required": ["id","featureType","isVirtual"]
    },

    "DualSpaceLayer": {
      "$anchor": "DualSpaceLayer",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["DualSpaceLayer"] },
        "creationDatetime": { "type": "string", "format": "date-time" },
        "terminationDatetime": { "type": "string", "format": "date-time" },
        "isLogical": { "type": "boolean" },
        "isDirected": { "type": "boolean" },
        "nodeMember": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/Node" },
          "uniqueItems": true
        },
        "edgeMember": {
          "type": "array",
          "items": { "$ref": "#/$defs/Edge" },
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","isLogical","isDirected","nodeMember"]
    },

    "Node": {
      "$anchor": "Node",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["Node"] },
        "duality" : { "type": "string", "format": "uri-reference" },
        "geometry" : { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/Point" },
        "connects" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "uniqueItems": true
        }
      },
```

```
      "required": ["id","featureType","duality"]
    },

    "Edge": {
      "$anchor": "Edge",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["Edge"] },
        "duality" : { "type": "string", "format": "uri-reference" },
        "weight" : { "type": "number" },
        "geometry" : { "$ref": "https://beta.schemas.opengis.net/json-fg/
geometry-object.json#/$defs/LineString" },
        "connects" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        }
      },
      "required": ["id","featureType","weight","connects"]
    },

    "InterLayerConnection": {
      "$anchor": "InterLayerConnection",
      "type": "object",
      "properties": {
        "id": { "type": "string" },
        "featureType": { "enum": ["InterLayerConnection"] },
        "connectedLayers": {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        },
        "connectedNodes" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        },
        "connectedCells" : {
          "type": "array",
          "items": { "type": "string", "format": "uri-reference" },
          "minItems": 2,
          "maximum": 2,
          "uniqueItems": true
        },
        "typeOfTopoExpression" : {
          "enum": ["CONTAINS", "OVERLAPS", "EQUALS", "WITHIN", "CROSSES",
"OTHERS"]
        },
        "comment" : { "type": "string" }
      },
      "required": ["id","featureType","connectedLayers","typeOfTopoExpression"]
    },

    "ExternalReferenceType": {
      "$anchor": "ExternalReferenceType",
      "type": "object",
```

```
      "properties": {
        "externalObject": {
          "type": "object",
          "properties": {
            "name": { "type": "string" },
            "uri": { "type": "string", "format": "uri" }
          },
          "required": ["name","uri"]
        },
        "informationSystem": { "type": "string", "format": "uri" }
      },
      "required": ["externalObject", "informationSystem"]
    }

  }
}
```

Listing B.1

B.2. JSON Schema of a IndoorJSON Navigation

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://www.opengis.net/spec/indoorgml/2.0/json/indoorjson.navi.schema.
json",
  "title": "IndoorJSON for IndoorGML 2.0 navigation part",
  "description": "IndoorJSON Navigation",
  "oneOf": [
    {"$ref": "#/$defs/NavigableSpace"       },
    {"$ref": "#/$defs/NonNavigableSpace"    },
    {"$ref": "#/$defs/GeneralSpace"         },
    {"$ref": "#/$defs/TransferSpace"        },
    {"$ref": "#/$defs/ObjectSpace"          },
    {"$ref": "#/$defs/NavigableBoundary"    },
    {"$ref": "#/$defs/NonNavigableBoundary" },
    {"$ref": "#/$defs/Route"                }
  ],
  "$defs": {

    "NavigableSpace": {
      "$anchor": "NavigableSpace",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellSpace"},
        {
          "type": "object",
          "properties": {
            "locomotionType": { "enum": ["Flying","Rolling","Walking",
"Unspecified"]}
          },
          "required": ["locomotionType"]
        }
      ]
    },

    "NonNavigableSpace": {
      "$anchor": "NonNavigableSpace",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellSpace"}
```

```
      ]
    },

    "GeneralSpace": {
      "$anchor": "GeneralSpace",
      "allOf": [
        {"$ref": "#/$defs/NavigableSpace"},
        {
          "type": "object",
          "properties": {
            "function": { "enum": ["Administration","Business, trade",
"Education, training","Recreation","Laboratory","Storage","Security"]}
          },
          "required": ["function"]
        }
      ]
    },

    "TransferSpace": {
      "$anchor": "TransferSpace",
      "allOf": [
        {"$ref": "#/$defs/NavigableSpace"},
        {
          "type": "object",
          "properties": {
            "category": { "enum": ["Door","Window"]},
            "function": { "enum": ["AnchorSpace","ConnectionSpace"]}
          },
          "required": ["category","function"]
        }
      ]
    },

    "ObjectSpace": {
      "$anchor": "ObjectSpace",
      "allOf": [
        {"$ref": "#/$defs/NonNavigableSpace"},
        {
          "type": "object",
          "properties": {
            "containedFeatures": { "type": "integer"},
            "description": { "type": "string"}
          }
        }
      ]
    },

    "NavigableBoundary": {
      "$anchor": "NavigableBoundary",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellBoundary"},
        {
          "type": "object",
          "properties": {
            "boundaryOrientation": { "type": "boolean"},
            "navigableBoundaryFunction": { "enum": ["AnchorBoundary",
"ConnectionBoundary"]}
          },
          "required": ["navigableBoundaryFunction"]
        }
      ]
    },
```

```
    "NonNavigableBoundary": {
      "$anchor": "NonNavigableBoundary",
      "allOf": [
        {"$ref": "indoorjson.core.schema.json#/$defs/CellBoundary"}
      ]
    },

    "Route": {
      "$anchor": "Route",
      "type": "object",
      "properties": {
        "creationDate": { "type": "string", "format": "date-time"},
        "routeNode": {
          "type": "array",
          "minItems": 2,
          "items": {"$ref": "indoorjson.core.schema.json#/$defs/Node"}
        },
        "routeEdge": {
          "type": "array",
          "minItems": 1,
          "items": {"$ref": "indoorjson.core.schema.json#/$defs/Edge"}
        }
      },
      "required": ["routeNode","routeEdge"]
    }
  }
}
```

Listing B.2

C

ANNEX C (INFORMATIVE) REVISION HISTORY

ANNEX C (INFORMATIVE) REVISION HISTORY

DATE	RELEASE	EDITOR	PRIMARY CLAUSES MODIFIED	DESCRIPTION
2026-06-03	0.1	Taehoon Kim	all	initial version

BIBLIOGRAPHY

BIBLIOGRAPHY

NOTE: The TC has approved Springer LNCS as the official document citation type.

Springer LNCS is widely used in technical and computer science journals and other publications

For citations in the text please use square brackets and consecutive numbers: [1], [2], [3]

Actual References:

[n] Journal: Author Surname, A.: Title. Publication Title. Volume number, Issue number, Pages Used (Year Published)

[1] ISO: ISO 19142, *Geographic information — Web Feature Service*. International Organization for Standardization, Geneva https://www.iso.org/standard/42136.html.

[2] W3C: **Data Catalog Vocabulary,** W3C Recommendation 16 January 2014, https://www.w3.org/TR/vocab-dcat/

[3] IANA: **Link Relation Types,** https://www.iana.org/assignments/link-relations/link-relations.xml

[4] W3C/OGC: **Spatial Data on the Web Best Practices,** W3C Working Group Note 28 September 2017, https://www.w3.org/TR/sdw-bp/

[5] W3C: **Data on the Web Best Practices,** W3C Recommendation 31 January 2017, https://www.w3.org/TR/dwbp/

[6] Ben-Kiki, O., Evans, C., Ingy döt Net: **YAML Ain't Markup Language,** https://yaml.org/

[7] OGC: **Web Feature Service 2.0,** http://docs.opengeospatial.org/is/09-025r2/09-025r2.html

[8] Berners-Lee, T., Fielding, R., Masinter, L.: **IETF RFC 3986 — Uniform Resource Identifier (URI): Generic Syntax,** http://tools.ietf.org/rfc/rfc3986.txt